Comparing and Evaluating Retrieval Augmented Generation (RAG) Strategies for a Code-Assisting Agentic CLI Tool

Zuri McKee
North Carolina A&T State University
Greensboro, North Carolina, USA
zcmckee@aggies.ncat.edu

Abstract

Most research on retrieval-augmented generation (RAG) for code focuses on large, single-language repositories and resource-heavy models. This study asks whether those findings hold for a setting common among students and hobbyist developers: a small, multi-language personal codebase and a locally hosted model on consumer hardware. Using a corpus of 95 files from ten personal projects in eight languages, we compare three conditions (no retrieval, BM25 lexical retrieval, and embedding-based semantic retrieval) with Qwen2.5-Coder 7B on fifteen repository-specific coding tasks of varying difficulty, for 45 runs in total. Embedding retrieval had the highest success rate (20%), followed by BM25 (13%) and the no-retrieval baseline (7%). Embedding retrieval also returned the target file most often (87% versus 67% for BM25) and met the most grading criteria. However, the model often failed even when it had the relevant code, fabricating methods and completing only part of multi-file edits. Retrieval also increased generation time by three to four times. These results suggest that for small local coding agents, the capability of the generation model limits performance more than retrieval does.

Keywords

Retrieval-Augmented Generation, Code Generation, Large Language Models, BM25, Embedding Retrieval, Local Models, Agentic Coding Tools

ACM Reference Format:
Zuri McKee. 2018. Comparing and Evaluating Retrieval Augmented Generation (RAG) Strategies for a Code-Assisting Agentic CLI Tool. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Retrieval-Augmented Generation, or RAG is a technique used to improve the generation or the output of a large language model (LLM). The purpose of RAG is to provide the LLM with additional context that is directly relevant to the set of tasks that it will undertake. This context is held in a knowledge base or a retrieval corpus. RAG optimizes an LLM by allowing it to be fine-tuned to the needs and activities of a corporation or developer without having to retrain the model [1].

Most research on this topic has been focused on large-scale, enterprise level documentation, and primarily with text-based tasks. However, there is a lack of study on RAG's effectiveness on small, frequently changing codebases with diverse retrieval sources. This scenario is becoming more and more common, especially as AI tools become more modular and accessible to hobbyist developers. Retrieval behavior could differ on a smaller scale, especially when retrieving from a smaller and more eccentric retrieval corpus.

This report will investigate whether existing conclusions on retrieval strategies for larger codebases can hold up at a smaller scale by comparing two retrieval strategies on a local Ollama coder model across a set of fifteen coding tasks directly related to the knowledge base. The coding tasks compose a test suite that contains fifteen repository-level coding prompts with corresponding expected output documents in the relevant coding language.

Scripts are used to facilitate file segmentation, indexing, and constructing embeddings. A harness script is also used to run all tasks with a human-judged retry loop. Lastly, a pass/fail grading script is run, where the results are passed into a script that will create tables on the data and write them to a Markdown file.

Three conditions will be tested against the retrieval strategies; Baseline no-retrieval, BM25 or lexical retrieval, and embedding or semantic retrieval. The model will be tested under each retrieval condition, so the 15 coding tasks will run 45 times total. The results will be analyzed to evaluate which retrieval method is the most successful.

2 Related Work

CodeRAG-Bench [5] is an evaluation benchmark with 9000 tasks and 25 million Python retrieval documents used to explore how RAG can be applied to diverse coding problems and retrieval sources. Through this study, the authors found that retrieval does not result in an immediate improvement for the models, and also saw that models were failing to find useful context and use said context effectively. However, they also found that in dense-retrieval models, larger models resulted in better retrieval, and that in repository-level challenges, all models improved, but only more powerful models had significant improvement. This demonstrates that retrieval enhances performance more when paired with a model with proficient code generation abilities (Wang et al., 2025).

RepoBench by Liu et al.,[3] took a different approach by separating repository-level code assistance into three evaluation tasks, focused on retrieval, code completion, and the complete process of a code auto-completion system. RepoBench studies cross-file context

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

retrieval for code generation using multiple retrieval methods and coder models to analyze the performance of the pipeline end-to-end, and to see which models succeed at managing extended context in real world coding environments. The RepoBench team found that Python file retrieval was consistently more accurate than Java file retrieval. They attribute this to the fact that typically function arguments are defined closely to their corresponding function calls, which could be helpful during retrieval, while Java's object-oriented design could complicate the retrieval process.

RepoCoder [6] uses an iterative RAG pipeline and a similarity based retriever to study the ability of automated code completion tools to find and use information scattered across whole repositories. RepoCoder's iterative loop allows the model's predictions to inform the consequent retrieval queries, allowing for more accurate repository-level context understanding, and also for the initial code predictions to serve as hints to help the model retrieve more relevant context in future iterations. They found that RepoCoder significantly improved the in-file completion baseline by more than 10% in all conditions, and that the iterative approach consistently performed better than regular RAG code completion. This proved that refined retrieval queries improve completion accuracy.

The results of this study will resemble the ones of the studies outlined above, but they will differ slightly because of some characteristics specific to this study. This report outlines the results of a study on a diverse, small codebase made up of less than 100 files in 8 different programming languages. Although many of the results found by the teams of CodeRAG-Bench, RepoBench, and RepoCoder are similar to the results of this study, the results and the implication of these results are different, because of the overall uniqueness of this subject. All of the studies within this literature review revolved around much larger knowledge bases that were monolingual, with different, more resource-heavy models and hardware that wouldn't be as accessible to a hobbyist developer. The results of this study will fill an existing gap, and will evaluate the performance of RAG methods on a small and diverse codebase.

3 Methodology

The retrieval corpus is composed of ten personal projects composing of 95 files total.[4] The projects range in complexity, in length, and in file type. The languages and frameworks represented are Python, Java, JavaScript, JavaScript-React, SQL, C++, HTML, and CSS. To prepare and clean the data, the knowledge base was stripped of all personal or identifying information, such as API keys or other private data. Additionally, all non-code related files were removed to conserve hardware and software resources, and to reduce noise while ensuring that the data was as relevant as possible.

To mitigate data leakage, I made sure to implement a test suite that was extremely specific to the corpus to ensure that the model's accuracy would fully depend on its knowledge of the codebase instead of its general coding ability. Also, the model used during testing was released in September of 2024, which predates all of the projects within the corpus, ensuring that this data was not in its training data.

3.1 Materials

All models used for this study were hosted locally on a Lenovo IdeaPad running Windows 11 on 16gb RAM and no GPU. The model used for code generation was the Ollama Qwen2.5-coder:7b, which was chosen because it is a lighter model and was more compatible with the present hardware and software, while still being a capable coding agent. The Ollama Qwen3-embedding:0.6b model was chosen to create the embeddings, and ChromaDB was used to store said embeddings. The BM-25 algorithm was used for lexical retrieval, and the Qwen3-embedding:0.6b model was used to create embeddings for semantic retrieval, which were two of the three conditions tested during the study. Along with the no-retrieval baseline, which was the Ollama model answering prompts using its default training.

3.2 Data Cleaning

After the retrieval corpus was fully cleaned and prepared, Claude Opus 5.5 was used to comb through the files to create the test suite. The test suite was composed of fifteen prompts, each matched with an expected output file. Some prompts used exact keywords found in relevant files in the corpus, while others used paraphrased language. The prompts specified the desired language for the output, and a manifest.json file detailed a list of target files for the model to reference for each prompt.

The test suite was specifically designed to resemble queries that a developer might pose to a model about a project, such as bug fixes, feature additions, or full refactors. The prompts were split into three categories; easy, medium, and hard. Easy prompts were composed of bug fixes or syntax errors, medium prompts were feature additions requiring the model to retrieve chunks from one to two existing methods or classes, while hard prompts were full refactors requiring extensive changes to a certain method/class or within multiple files. The difficulty range of the test suite helped to pose the question; Does retrieval assist more on more difficult, multi-file dependent tasks, or with easier, single-file tasks?

3.3 Preparation

To prepare the retrieval pipeline, a set of scripts were used to chunk, index, build the embeddings, and test the retrieval methods by hand. Whole-file chunking was the default, since most files within the corpus were relatively short. If the file was over 200 lines, it was chunked by top-level functions or class boundaries. This allowed for both of the retrieval conditions to have the same chunk boundaries, so that differences were concentrated within the retrieval methods, not from one method receiving smaller chunks than the other.

To ensure that all three retrieval strategies would be examined under the same conditions, all other variables were kept constant with the only independent variable being the retrieval strategy. The same generation model and version were used throughout testing, along with the test suite, model generation settings, chunks, retry limit, success criteria, and the hardware and software environments.

3.4 Testing and Grading

During testing, prompt feasibility was judged by hand using a harness that fed the prompts to the model by parsing a manifest.json file. The harness would output a response, and I would review it and

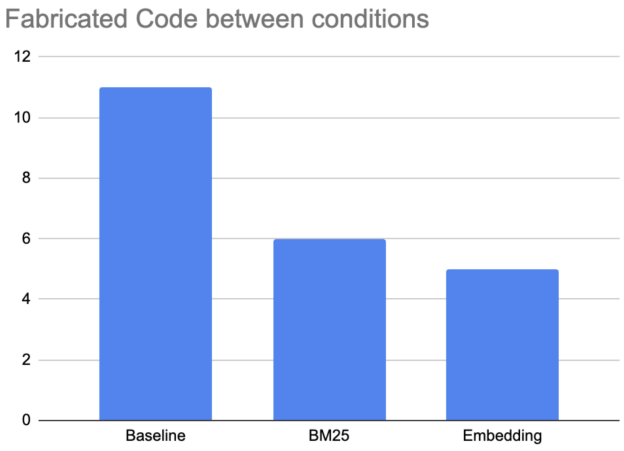


Figure 1: Instances of fabricated code by retrieval condition.

make sure that it was robust enough and directly relevant to the current prompt. If the model's response was thin or irrelevant, it was retried, and if the model exceeded three attempts on a prompt, that prompt would automatically be marked as a failure and was marked to have hit the retry cap.

Model responses were graded pass/fail against a set of grading criteria after running the full test suite. Each model's final response was graded against an expected output written before testing. A response was marked as successful if it implemented the required behavior in the target files. After testing for each prompt under each condition, the task success rate by difficulty, prompt style, and retrieval was calculated for each condition. This will serve as one of the primary data points. While the retry metric was supposed to be one of the main data points, no runs hit the retry cap and only two had to be rerun more than once, making it useless. To arrive at a more substantial conclusion I will analyze more relevant data points such as instances of fabricated code between conditions, number of criteria passed per task, and if the successful cases retrieved their target file.

3.5 Limitations

Although testing was successful, some limitations were recognized while analyzing results. The hardware used to perform testing and to host the models was sub-par, gradually becoming unable to handle the load of the local models while running the test harness. This destabilized the testing environment by causing occasional crashes and incredible latency during retrieval tasks. The BM25 and embedding tasks were the slowest, with BM25 performing almost four times slower than the baseline and embedding being three times slower. This could be attributed to the hardware resource bottleneck, or to the retrieved chunks added on to the model's prompt. Additionally, the retrieval queries were very short. The harness searched using only the task title that was stated in the manifest file, not the full prompt, which could have given the retrieval conditions less to work with, resulting in less chunks or lower relevance in chunks returned.

Table 1: Successful Runs and Target File Retrieval

Task ID	Condition	Retrieval Style	Relevant Chunks	Target File
2	BM25	paraphrased	2	No
2	Embedding	paraphrased	2	Yes
5	BM25	keyword	3	Yes
5	Embedding	keyword	1	Yes
11	Embedding	paraphrased	1	Yes

4 Results

The success rate between the three conditions, baseline-no-retrieval, BM25 keyword, and embeddings barely differed. The embedding tasks performed best at 20%, while BM25 and baseline were not far behind at 13% and 7% respectively. Although these numbers may seem distinct, the difference between each condition was only one task. Additionally, embedding retrieval also beats keyword in retrieving target files. Embeddings retrieved target files 13 times out of 15, while BM25 retrieved them 10 times. The success rates between difficulty levels were minute.

The baseline condition only succeeded on one question in the easy category, BM25 was successful for two easy prompts, embedding was also successful for two easy prompts and one hard one. No medium difficulty prompts were successful under any condition. In terms of the success rate by prompt style, overall, all three conditions performed better on paraphrased prompts than with keyword prompts. Baseline succeeded on 0/8 prompts for keyword versus 1/7 for paraphrased, BM25 succeeded on 1/7 prompts for paraphrased and 1/8 for keyword, while embedding achieved 2/7 for paraphrased and 1/8 for keyword. When examining the success of the retrieval methods, I found that more often than not, the model retrieved the relevant code and still failed. This leads me to conclude that generation is the bottleneck, not retrieval. For example, in Task 9, the model retrieved the exact chunk that it needed for a specific prompt, but it still hallucinated code and fabricated methods.

Fabricated code was another common issue found among the results. Keyword retrieval had fabricated code in over a third of its answers. Additionally, the data demonstrated that retrieval increased latency and generation time, with little increase of accuracy or successes in return. However, although overall success was very low, there are some indicators of retrieval producing more optimal results than non-retrieval strategies. The embedding condition had the greatest number of pass criteria met out of any of the other conditions, meeting almost 50 out of 76 pass criteria. Additionally, within the six successful runs, five used retrieval, and four of the five successful runs returned their target file.

5 Discussion

On first glance, the overall results returned from this study reflect massive failure among all three conditions. However, the data and overall variables in the study suggest otherwise, and can be used to analyze the results further. The high number of failed prompts could be attributed to a failure to retrieve relevant information or model hallucination. Many of the model's answers referenced methods that didn't exist, and stated "assuming you have" in reference to a specific line or method. This suggests that the model didn't have the needed file in its context. For example, for Task 10 the model

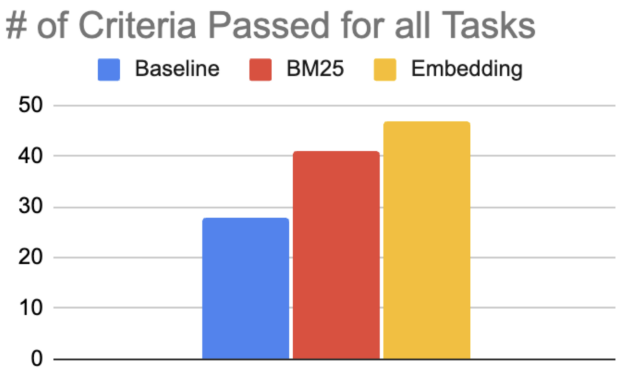


Figure 2: Number of criteria passed for all tasks per condition

retrieved the modules needed for the prompt, but not the main app.py file meant to call the defined modules, causing the model to fabricate app.py and guess the module imports from the chunks that it did retrieve.

On the other hand, some model responses retrieved the right chunks, but didn't reason correctly, causing the model to miss root causes for bug fixes and to fabricate code. These failures point to the limitations of the generation model, and could be remedied by using a stronger one. The results also proved that multi-file edits fail more often than single-file edits. The hard tasks required coordinated changes in 4-6 places, and every response handled one or two of the changes while ignoring the rest and declaring it to be done. This could be seen in tasks 9, 13, and 14, which were all focused on multiple Java files, where every response handled one or two changes and missed the others.

However, some favorable responses were seen in tasks 2, 5, and 11, where at least 2/3 conditions succeeded on each of these tasks. Task 5 was in Python and required the model to create a new file and import and wire it into the main app.py file. The baseline condition fabricated app.py completely, but both retrieval conditions were able to retrieve all relevant files, and edit app.py without breaking it or fabricating code, showing that under the right conditions, retrieval can make a difference.

5.1 Future

In future iterations of this study, there are some variables that I would alter. This is because the results have demonstrated that a different approach would be necessary to obtain more substantial data and to come to a more satisfying conclusion. Running the prompts more than once per condition with different code generation and embedding models could widen the breadth of data returned, allowing for less generalization and more effective data analysis.

Additionally, both passes of the prompt grading process could be remedied by integrating unit tests instead of human review. During the first pass while running the test suite, instead of going over each prompt visually, a series of unit tests should be used for quality control. This could lead to higher success rates as well, because the unit tests could weed out fabricated output or output that doesn't compile before the second grading pass. This implementation would also make the retry metric a more interesting data point by allowing comparison of which conditions or tasks error-ed out and hit the retry limit the most.

6 Closing

Most RAG research does not cover small, hobbyist codebases. My goal is that these results could be helpful to a student or solo developer that is working to host their own local models and wants to integrate RAG. Although these results did demonstrate that retrieval has a small effect on the accuracy of code-generation tasks, those results are specific to the variables of this study, so differing results could be found with further experimentation. For example, embedding retrieval found the target file in 87% of runs but only succeeded 20% of the time. This points to generation model limitations.

In the future, I would test with a larger and stronger model on better hardware, add automated unit tests for each task to make grading more deterministic, and run each task several times to measure variance and for a wider breadth of data. Overall, retrieval improved task success for a small local coding agent, with embedding retrieval performing best, but the gains were modest and came at a significant latency cost. For solo developers working with small codebases, these results suggest that model capability, not retrieval, is the main bottleneck.

Acknowledgments

Generative AI Tools were used to assist with writing portions of the test harness and evaluation scripts, drafting acceptance criteria, and analyzing returned data. All AI-assisted content was reviewed and verified by the author.

Ethics and Privacy Statement

Data. The knowledge base consists exclusively of the author's own personal projects (about 75 source files across 8 repositories). No third-party, proprietary, or personal user data was included. Before indexing, all files were scanned and any secrets, API keys, or credentials were redacted. One projects was built from a publicly available Streamlit template; this is attributed to its original source.

Local execution. All models (Qwen2.5-Coder 7B and Qwen3 Embedding) were run locally through Ollama. No code, prompts, or model outputs were sent to external services during the experiment.

Human subjects. This study did not involve human participants. The author was the only person who designed the tasks and graded model outputs, so Institutional Review Board (IRB) review was not required.

Licensing and reproducibility. The source code, task definitions, acceptance criteria, and results are publicly available in the project repository [4]. The models used are openly licensed.

Limitations and potential harms. Because the corpus reflects a single developer's coding style, the findings should not be generalized to other codebases without further testing. AI coding agents can produce code that looks correct but contains errors; this study's low success rates highlight the importance of human review before using AI-generated code.

References

[1] Amazon Web Services, Inc. n.d.. *What is RAG? - Retrieval-Augmented Generation AI Explained.* Retrieved October 17, 2026 from https://aws.amazon.com/what-is/retrieval-augmented-generation/

[2] Anthropic. 2026. *Claude.* Retrieved October 17, 2026 from https://claude.ai

[3] Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. Repobench: Benchmarking repository-level code auto-completion systems. In *International Conference on Learning Representations*, Vol. 2024. 47832–47850.

[4] Zuri McKee. 2026. *agenticliragproj: Comparing RAG Strategies for a Code-Assisting Agentic CLI Tool.* Retrieved October 17, 2026 from https://github.com/zurimckee/agenticliragproj

[5] Zora Zhiruo Wang, Akari Asai, Frank F Xu, Yiqing Xie, Graham Neubig, Daniel Fried, et al. 2025. Coderag-bench: Can retrieval augment code generation?. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 3199–3214.

[6] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2471–2484.

Received 07 October 2026; revised N/A; accepted N/A